

JAVA-OOP CONCEPTS

Class & Object

Summary: A class is a blueprint or template used to create objects. It contains variables (data members) and methods (functions).

An object is an instance of a class that represents real-world entities. Objects are used to access the properties and methods of a class.

Program :

```
class Student {  
  
    String name = "Girishma";  
    int age = 22;  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
  
    public static void main(String[] args) {  
  
        Student s = new Student(); // Object creation  
        s.display();  
  
    }  
}
```

Output:

Name: Girishma

Age: 22

```
C:\Users\SLA 1035 Girishma K\Desktop>javac Students.java  
C:\Users\SLA 1035 Girishma K\Desktop>java Students.java  
Name: Girishma  
Age: 22
```

Main Concepts of OOP:

- 1.Encapsulation
- 2.Inheritance
- 3.Ploymorphism
- 4.Abstraction

1. Encapsulation:

Encapsulation means wrapping data and methods into a single unit called a class. It helps to protect data.

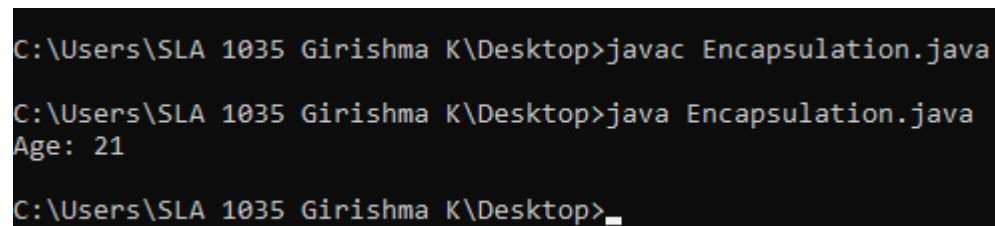
Program:

```
public class Encapsulation {  
  
    private int age = 21;  
  
    public void display() {  
        System.out.println("Age: " + age);  
    }  
  
    public static void main(String[] args) {
```

```
    Encapsulation obj = new Encapsulation();  
    obj.display();  
}  
}
```

Output:

Age:21



```
C:\Users\SLA 1035 Girishma K\Desktop>javac Encapsulation.java  
C:\Users\SLA 1035 Girishma K\Desktop>java Encapsulation.java  
Age: 21  
C:\Users\SLA 1035 Girishma K\Desktop>_
```

2. Inheritance:

Inheritance allows one class to acquire the properties and methods of another class. It helps in code reusability.

Program:

```
public class Inheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
        d.bark();  
    }  
}
```

```
class Animal {  
    void sound() {
```

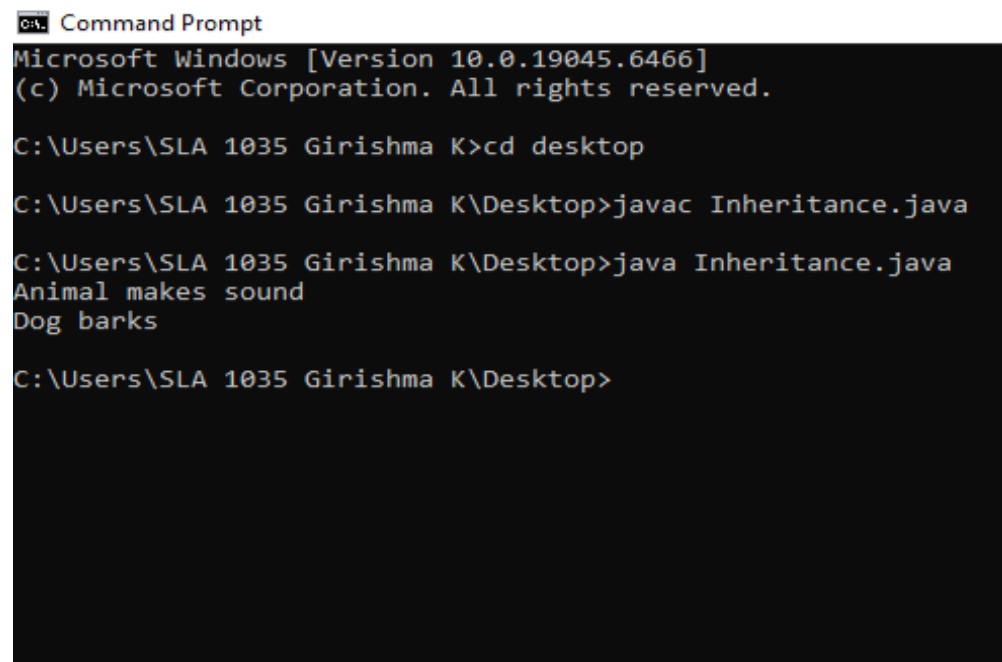
```
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

Output:

Animal makes sound

Dog barks

A screenshot of a Windows Command Prompt window. The title bar reads "C:\> Command Prompt". The window content shows the following text: "Microsoft Windows [Version 10.0.19045.6466] (c) Microsoft Corporation. All rights reserved. C:\Users\SLA 1035 Girishma K>cd desktop C:\Users\SLA 1035 Girishma K\Desktop>javac Inheritance.java C:\Users\SLA 1035 Girishma K\Desktop>java Inheritance.java Animal makes sound Dog barks C:\Users\SLA 1035 Girishma K\Desktop>". The text is displayed in a monospaced font on a black background with white text.

```
C:\> Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.
C:\Users\SLA 1035 Girishma K>cd desktop
C:\Users\SLA 1035 Girishma K\Desktop>javac Inheritance.java
C:\Users\SLA 1035 Girishma K\Desktop>java Inheritance.java
Animal makes sound
Dog barks
C:\Users\SLA 1035 Girishma K\Desktop>
```

3. Polymorphism:

Polymorphism means one method can perform different actions. It is achieved through method overloading and method overriding.

Program:

```
public class Polymorphism {

    public void display() {
        System.out.println("This is Parent Class");
    }

    public static void main(String[] args) {
        Child obj = new Child();
        obj.display();
    }
}

class Child extends Polymorphism {

    public void display() {
        System.out.println("This is Child Class");
    }
}
```

Output:

This is Child Class

```
C:\Users\SLA 1035 Girishma K\Desktop>javac Polymorphism.java
C:\Users\SLA 1035 Girishma K\Desktop>java Polymorphism.java
This is Child Class
```

4. Abstraction:

Abstraction hides implementation details and shows only important information to the user.

Program:

```
public abstract class Abstraction {  
  
    abstract void display();  
  
    public static void main(String[] args) {  
        Demo obj = new Demo();  
        obj.display();  
    }  
}  
  
class Demo extends Abstraction {  
  
    void display() {  
        System.out.println("Abstract Method Implemented");  
    }  
}
```

Output:

Abstract Method Implemented

```
C:\Users\SLA 1035 Girishma K\Desktop>javac Abstraction.java  
C:\Users\SLA 1035 Girishma K\Desktop>java Abstraction.java  
Abstract Method Implemented
```

